

Preliminary Design Document

Sentinel Version 1.0

Bryan Zhang, Bruce Zhang

Tuesday, March 31, 2026

Table of Contents

1. INTRODUCTION	3
1.1 Project Overview	3
2. PRODUCT DESIGN	3
2.1 Block Diagram	4
2.2 Sub-Circuit Descriptions	5
2.2.1 Circuit #1 Tank	5
2.2.1.1 Sub-Circuit #1 Tank Power Supply	5
2.2.1.2 Sub-Circuit #2 Tank Tracks	8
2.2.1.3 Sub-Circuit #3 Tank Flywheel Shooting	9
2.2.1.4 Sub-Circuit #4 Tank Turret Rotation	10
2.2.1.5 Sub-Circuit #5 Tank Dart Shover	12
2.2.1.6 Sub-Circuit #6 Tank Barrel Elevation	12
2.2.1.7 Sub-Circuit #7 Tank Laser Beam	13
2.2.1.8 Sub-Circuit #8 Tank Vision Processing	14
2.2.2 Circuit #2 Tank-Controller Interface	15
2.2.2.1 Sub-Circuit #9 Tank-Controller Live Video	15
2.2.2.2 Sub-Circuit #10 Tank-Controller Radio	17
2.2.3 Circuit #3 Controller	19
2.2.3.1 Sub-Circuit #11 Controller Power Supply	19
2.2.3.2 Sub-Circuit #12 Controller Gamepad	20
2.3 Product Interfaces	23
2.3.1 Controller Internal Interfaces	23
2.3.1.1 Interface #1: Arduino UNO R4 (U2) to USB Host Shield (U3)	23
2.3.1.2 Interface #2: Arduino UNO R4 (U2) to nRF24L01 Adapter (U4) to nRF24L01+PA+LNA (U6)	24
2.3.1.3 Interface #3: Power (Battery BT1 to Arduino U2)	25
2.3.2 Controller External Interfaces	25
2.3.2.1 Interface #4: Xbox Controller (U5) to USB Host Shield (U3)	25
2.3.2.2 Interface #5: Arduino Programming (U2)	25
2.3.3 Tank Internal Interfaces	25
2.3.3.1 Interface #6: Arduino UNO R4 (U11) to Motor Drivers (U14, U17)	25
2.3.3.2 Interface #7: Arduino UNO R4 (U11) to ULN2003 (U8) and BYJ-48 (M2)	26
2.3.3.3 Interface #8: Arduino UNO R4 (U11) to Servo Motors (M1, M3)	27
2.3.3.4 Interface #9: Arduino UNO R4 (U11) to nRF24L01 Adapter (U13) to nRF24L01+PA+LNA (U15)	27
2.3.3.5 Interface #10: Arduino UNO R4 (U11) to Size-130 DC Motors (M6, M7) via MOSFET (Q1)	28
2.3.3.6 Interface #11: Arduino UNO R4 (U11) to Laser Module (LD1)	29
2.3.3.7 Interface #12: Arduino UNO R4 (U11) to Raspberry Pi 4 (U7)	29
2.3.3.8 Interface #13: Power Circuit (BT2 to F1 to SW1 to Distribution)	29
2.3.3.9 Interface #14: FPV Camera (U12) and VTX Transmitter (U9)	30
2.3.4 Tank External Interfaces	31
2.3.4.1 Interface #15: Raspberry Pi 4 (U7) - Webcam (U10)	31
2.3.4.2 Interface #16: Arduino Programming (U11)	31
2.3.4.3 Interface #17: Raspberry Pi Programming (U7)	31
2.3.5 Controller-Tank Interfaces	32
2.3.5.1 Interface #18: FPV Video Downlink to FPV Monitor	32
2.3.5.2 Interface #19: nRF24L01 Wireless Link	32
3. SCHEDULING	32
3.1 Tasks	32
3.2 Gantt Chart Schedule	33
APPENDIX	34
Appendix A. Schematic	34
Appendix B. Parts List	36
Appendix C. Resources	39

1. Introduction

1.1 Project Overview

The Sentinel is a remotely operated robotic toy tank designed to combine entertainment and practical engineering in a single platform. Intended for hobbyists, students, and anyone curious about robotics, the Sentinel replicates many features of a real tank: rubber-tracked drive powered by two independent 6V JGB37-550 DC gear motors, a continuously rotating turret driven by a 28BYJ-48 stepper motor, an MG996R servo-controlled barrel with adjustable elevation, and a flywheel-based Nerf dart launcher capable of firing at up to 2 darts per second. The tank is constructed primarily from 3D-printed PLA and PETG components, housing all electronics within a compact chassis powered by a single 7.4V 2S Li-ion battery.

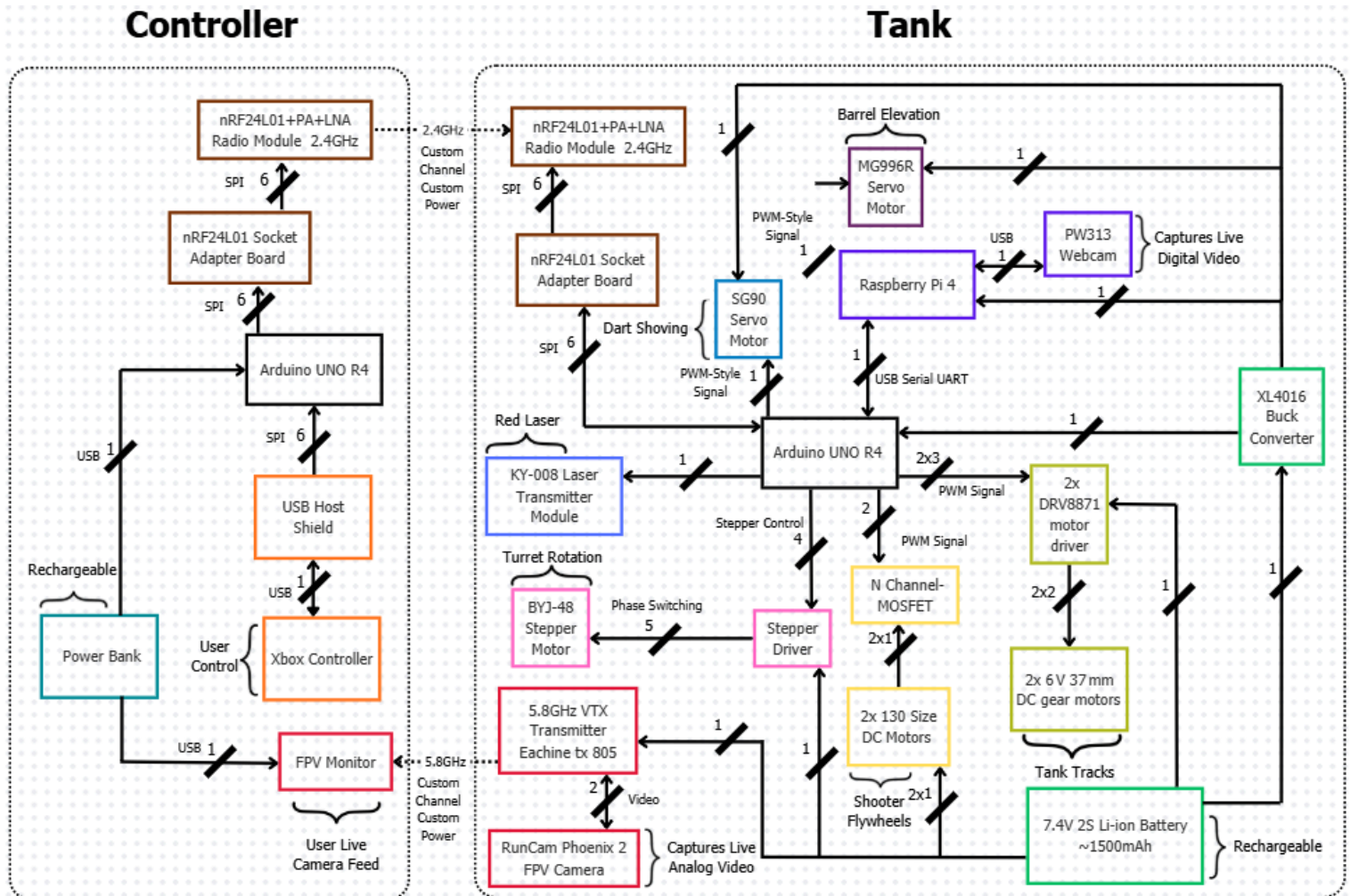
The operator controls the tank through a wired Xbox One controller attached to a custom 3D-printed handheld controller unit, which communicates wirelessly with the tank over a 2.4 GHz nRF24L01+PA+LNA radio link. Simultaneously, the operator receives a live first-person video feed transmitted wirelessly at 5.8 GHz from a RunCam Phoenix 2 FPV camera mounted on the tank to a SoloGood FPV monitor attached to the controller — allowing the tank to be driven effectively from far away and out of direct line of sight. The controller unit is built around an Arduino UNO R4 microcontroller paired with a USB Host Shield, while the tank itself is controlled by a second Arduino UNO R4 responsible for all motor, servo, and actuator outputs.

Beyond manual operation, the Sentinel also features a semi-autonomous targeting mode powered by a Raspberry Pi 4 onboard the tank. Using a PW313 USB webcam and the OpenCV computer vision library, the Pi continuously processes live video frames to detect and track human targets in real time using a MobileNet-SSD detection model and SORT tracking algorithm. When autonomous mode is activated, the Pi commands the Arduino to rotate and elevate the turret toward the nearest identified target automatically, while still requiring the operator to manually pull the trigger to fire — preventing unintended behaviour. A red KY-008 laser module mounted on the barrel further aids aiming precision and can be toggled on and off by the operator at any time.

A full Hardware Requirements Specification (HRS) was written prior to this document and is available for reference, detailing all functional, hardware, and interface requirements agreed upon with the project stakeholder. This Preliminary Design Document (PDD) builds on those requirements with detailed sub-circuit descriptions, interface definitions, component-level calculations, and a project schedule covering assembly through to final testing.

2. Product Design

2.1 Block Diagram



2.2 Sub-Circuit Descriptions

2.2.1 Circuit #1 Tank

2.2.1.1 Sub-Circuit #1 Tank Power Supply

The tank's power supply is designated around a single rechargeable 7.4 V (2S) Li-ion battery, which

provides the power to all subsystems on the tank, including the drive motors, the control electronics, the turret system and the laser module.

The rechargeable 7.4 V 2S Li-ion battery connects to the rest of the circuit. Through a SS24 Toggle switch, the user can turn on and off power to the tank circuit. For safety, through a replaceable 15 A fuse, the circuit will break if the current exceeds 15 A to prevent damage to other circuit components.

A XL4016 8 A buck converter is used in the power sub-circuit to step down the battery voltage from 7.4 V to a stable 5 V, which is required by both the Arduino and the Raspberry Pi. These components are sensitive to voltage levels, and supplying them directly with the battery voltage would damage them or cause unstable operation. The buck converter ensures a consistent output voltage even when the battery voltage fluctuates under load, such as when the motors are running. Additionally, buck converters are highly efficient (typically 85–95%), meaning very little energy is lost as heat compared to linear regulators. This improves overall system efficiency and helps extend battery life. By isolating the low-voltage electronics from the high-current motor supply and providing both stable voltage and adequate current, the buck converter contributes to a more stable and reliable control system.

To ensure the battery and buck converter can meet system demands, the current draw of each major component is estimated and summed.

Part	Current Estimation	Source	Quantity	Rated Voltage (v)
6 V 56:1 37 mm JGB37-550 DC motor	Stall Current: 3 A	Source	2	6
130 size high RPM DC motors	Stall Current: 500 mA max	Source	2	4.5 - 9
Arduino UNO R4	Current: 97 mA - 101 mA	Source	1	5
SG90 Servo	Current max: < 600 mA	Source	1	4.8 - 6
MG996R Servo	Running Current: 500 mA Stall Current: 2.5 A	Source	1	4.8 - 7.2
Raspberry Pi 4	Recommended Current: 3 A	Source	1	5
AverMedia PW313 webcam	Max current: 500 mA (USB 2.0)	Source	1	5
5.8GHz VTX Transmitter Eachine tx 805 +	VTX Transmitter Current:	Source	1	7 - 24

	800 mW: around 350 mA			
Runcam Phoenix 2 FPV Camera	Camera Running Current: around 200 mA	Source	1	5-36
BYJ-48 Stepper Motor	300 mA during startup or load changes	Source	1	Minimum 5
KY-008 laser module	20-40 mA	Source	1	5

The overall current draw of the system can be estimated by summing the consumption of all subsystems and grouping them by their operating voltage rails. The 5 V logic rail is the most critical in terms of stability, as it powers the Raspberry Pi 4, Arduino UNO R4, AverMedia PW313 webcam, MG996R servo, SG90 servo, nRF24L01+PA+LNA, and BYJ-48 stepper motor. Their combined current draw is approximately 3 A (Pi) + 0.1 A (Arduino) + 0.05 mA (nRF24L01+PA+LNA in RX mode) + 0.2–0.5 A (webcam) + 0.3 A (stepper) + 0.3–0.6 A (SG90 servo) + 0.6–1.5 A (MG996R servo), giving a subtotal of 4.5–6 A at 5 V. Because the XL4016 8 A Buck Converter has a max current rating of 8 A, it can sufficiently power all the 5 V components with additional headroom to ensure reliable operation and prevent overheating or brownouts.

The actuator and motor subsystems represent the largest current demand in the system because of their high torque and speed requirements. The two 6 V JGB37-550 37 mm DC gear motors draw at max approximately 6 A combined, while the two 130-size DC motors contribute approximately 1 A total. The MG996R servo adds up to 2.5 A in stall conditions, and the SG90 servo contributes up to 0.6 A peak current. When combined, these high-current motors draw approximately 5-6 A under typical operation and up to 10 A in worst-case conditions, particularly during motor startup or servo stall events. These transient spikes are important for design considerations, as they can cause voltage dips if the power distribution system is undersized.

The FPV and transmission subsystem adds a smaller but continuous load. The Eachine TX805 VTX draws approximately 350 mA at 800 mW transmission power, while the RunCam Phoenix 2 camera draws around 200 mA. Combined, this subsystem contributes approximately 0.6–1 A of continuous current draw. Although relatively small compared to the motors, it remains significant due to its constant operation and sensitivity to voltage instability.

Overall, the system is expected to draw approximately 9–10 A during normal operation, with a potential peak demand of 14 A under worst-case simultaneous loading conditions across motors, servos, and processing hardware.

The system is powered by a single 7.4 V (2S) Li-ion battery, which serves as the main energy source for all subsystems. The battery itself can sustain the heavy current draw; however, it will likely have a short battery life. Given the 7.4 V 5000 mAh 18650 2S Li-ion battery, the max capacity is 5000 mAh and the discharge rate is 20 C, so the max current can be calculated with the following:

$$\text{Max current} = \text{Capacity} \times \text{Discharge C Rating} \approx 5 \text{ Ah} \times 20 \text{ C} \approx 100 \text{ A}$$

The max current the battery can source being 100 A is well above the estimate of how much current we need to draw; however, to prevent accidental damage to any components in case high current is drawn, a 15 A fuse is placed right next to the battery.

However, since different components require different operating voltages, a regulated power distribution system is necessary to ensure safe and stable operation.

A 5 V regulated rail is required for all logic and low-voltage electronics, including the Raspberry Pi 4, Arduino UNO R4, nRF24L01+PA+LNA radio module, webcam, and BYJ-48 stepper motor. These components are highly sensitive to voltage fluctuations, particularly the Raspberry Pi, which requires a stable 5 V supply and can experience data corruption or resets if voltage drops occur. Additionally, the two servo motors, which are also rated at around 4.8–6 V, will be using this 5 V power, as being powered directly from the Arduino 5 V would damage the Arduino due to its current draw and being powered directly from the 7.4 V Li-ion cell would damage the servos. A high-current buck converter, the XL4016 8 A Buck Converter, is therefore used to step down the 7.4 V battery supply to a stable 5 V output, ensuring consistent operation even under fluctuating motor loads.

A separate 7.4 V rail is required for the DC motors because of their higher voltage requirement. The 37 mm gear motors are rated at 6 V, while the size-130 motors are assumed to function with 3–9 V. This rail must be capable of delivering high transient currents, particularly during motor startup and servo stall events, so all of these components are directly powered from the battery. The FPV system operates partially outside the regulated rails. The Eachine TX805 VTX accepts a wide input range of 7–24 V and is therefore powered directly from the 7.4 V battery supply. This reduces conversion losses and simplifies power distribution for the high-frequency transmission system. The RunCam Phoenix 2 camera can operate either from 5 V or a higher voltage range (up to 36 V), but in this system it is powered through the VTX's 5 V output.

Overall, the power architecture is structured around a single battery source with multiple regulated outputs. The separation into 5 V logic and low-voltage motors and 7.4 V heavy-duty actuation ensures that high-current motor loads do not interfere with sensitive electronics, improving system stability, reducing electrical noise, and maintaining consistent performance across all subsystems.

2.2.1.2 Sub-Circuit #2 Tank Tracks

The tank tracks subcircuit is responsible for controlling the movement of the tank. The sub-circuit includes two 37 mm JGB37-550 metal gear DC motors and two DRV8871 motor drivers to control the speed and direction of the motors.

The DRV8871 motor driver contains a single H-bridge, which allows control of one motor. Each motor is connected to its own driver module, with the outputs supplying current directly from the battery.

IN1/OUT1	IN2/OUT2	Result for Motor
LOW	LOW	Coast/Off
LOW	HIGH	Spin Backward
HIGH	LOW	Spin Forward

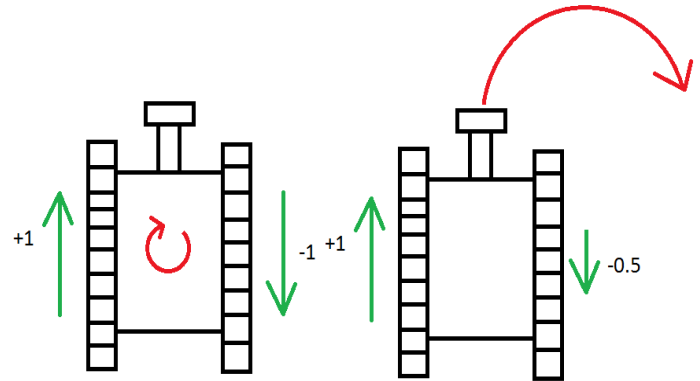
HIGH	HIGH	Brake
------	------	-------

Direction control is achieved using the input pins of the motor driver. Because both output pins connect to the positive and negative terminals of the motor, setting the driver pins HIGH or LOW offers directional controls, allowing the motors to change direction, brake, and coast. Note that because all motors are identical (no such thing as left or right motors), one of the two drivetrain motors will be inverted through software as they are mirrored (pointed in opposite directions, so they spin in reverse relative to each other).

Since each motor can be controlled independently, the tank can turn by varying the motion of each tread. For example, to turn right on the spot, the tank drives the left tread forward and the right tread in reverse. Speed control of the tank is achieved by using PWM signals, which are applied to the motor driver's PWM input.

$$V_{avg} = Duty\ Cycle \times V_{supply}$$

PWM controls the speed of the motors by varying the average voltage delivered to the motors. Using PWM, the tank can turn while moving by slowing down tracks. For example, the tank can turn right while moving forward by slowing down the right tank tread.



The torque required to move the tank depends on its mass. Taking into account the components like the motors, battery, Nerf elements, Arduino, Raspberry Pi, Cameras, and the PLA tank chassis the estimated weight of the tank is around 2.3kg. The force required to move the tank is distributed between the two motors reducing the load on each motor. The torque requirement per motor can be estimated with the following:

$$\tau = r \times F$$

Where τ is the torque, r is radius of the drive wheel (around 3 cm), and F is the force required to move the tank

The force can be estimated using

$$F = m \times g \times \mu \approx 2.3\text{ kg} \times 9.81\text{ m/s}^2 \times 0.5 \approx 11.3\text{ N}$$

Where m is the mass, g is the gravitational acceleration (9.8 m/s^2), and μ is the friction coefficient (around 0.5 for rubber tracks on a smooth surface)

Therefore, the required torque is:

$$\tau = 0.03\text{ m} \cdot 11.3\text{ N} \approx 0.34\text{ N}\cdot\text{m}$$

Since each motor only provides half the total driving force, the required torque per motor is approximately $0.17\text{ N}\cdot\text{m}$ under ideal conditions. However, in practice this number should be multiplied by 2-3 times because of real world variables such as starting friction, turning resistance, and different driving surfaces. This line of reasoning results in the requirement that each motor should have a torque of at least $0.51\text{ N}\cdot\text{m}$, or around $5\text{ kgf}\cdot\text{cm}$, for safe measures. As such, the 6 V 56:1 37 mm JGB37-550 DC motors, which have a rated torque of $5.0\text{ kgf}\cdot\text{cm}$, are perfect for this application.

The drive subsystem uses two 6 V 37 mm DC gear motors (≈ 100 RPM, ~ 5 kg·cm usable torque) paired with two DRV8871 motor driver modules. Each driver safely handles up to 3.6 A per motor and supports PWM speed control and bidirectional operation. Fortunately, the JGB37-550 DC motors draw up to 3 A at stall, falling within this requirement. This combination provides sufficient torque and electrical compatibility while remaining cost-effective and within the current limits of the control system.

Additionally, the system is powered by a 7.4 V battery; however, due to the very low internal voltage drop of approximately 0.2 V across the DRV8871 motor driver, the effective voltage supplied to the motors is nearly equal to the battery voltage, around 7.2 V at stall current. By selecting a motor rated for 6 V operation, the motor still operates safely within its tolerance range. The minimal voltage drop ensures the motor receives nearly full battery voltage, allowing efficient performance, responsive acceleration, and sufficient torque output without significant power loss.

2.2.1.3 Sub-Circuit #3 Tank Flywheel Shooting

The tank's flywheels are the key part that allows the tank to serve its primary function of shooting Nerf darts. This subcircuit can be split into two main parts: the motor drive circuit and the Arduino PWM control circuit.

The flywheel mechanism consists of two 130-size high-RPM DC motors, each attached to a 3D-printed PLA flywheel. These motors are connected in parallel, ensuring that both motors receive the full supply voltage from the battery. This is important because DC motor speed is approximately proportional to the applied voltage:

$$\omega \propto V$$

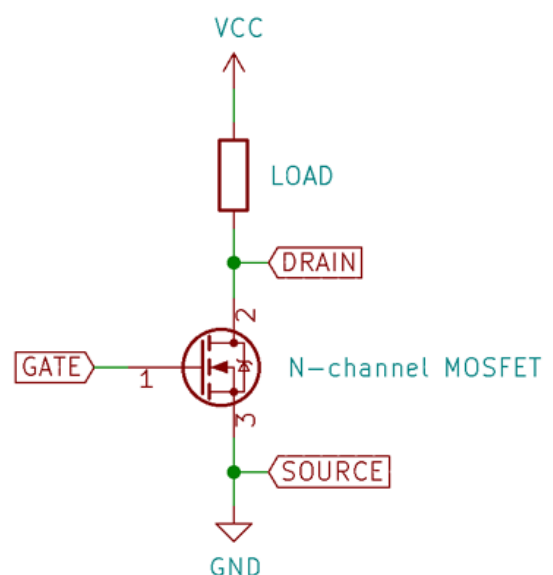
$$\omega = \text{rotational speed}$$

$$V = \text{voltage}$$

By wiring the motors in parallel, both motors operate at maximum designed speed, which is necessary to maintain sufficient flywheel velocity to launch a Nerf dart.

The positive terminals of both motors are connected to the battery positive. The negative terminals are connected to the Drain of an N-Channel MOSFET (IRLZ44N). A MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor), in this case, acts as an electronic switch that allows a low-power control signal to turn a higher-current device, like a motor, on or off. A flyback diode is placed across the motors to protect the MOSFET from voltage spikes caused by the motors. When the motors are suddenly turned off they can produce a reverse voltage spike, which the diode safely deals with.

When voltage is applied through the MOSFET Gate, a conductive channel is created between the Drain and Source, allowing current to flow through the motors. The MOSFET is chosen because our design does not require bidirectional control, which makes the MOSFET the more efficient option over a H-Bridge Motor Driver. The MOSFET



is also directly compatible with Arduino PWM signals and generates less heat while still being able to handle the high current required by the motors.

The Gate of the MOSFET is connected to an Arduino Uno R4 PWM pin through a 220Ω resistor. This resistor limits current to the Gate and protects the Arduino. A 10KΩ pull down resistor is then connected between the Gate and ground to ensure that the MOSFET remains OFF when no signal is present. This prevents unintended motor activation due to a floating gate.

The Arduino controls motor speed using Pulse Width Modulation (PWM).

$$V_{avg} = \text{Duty Cycle} \times V_{supply}$$

PWM rapidly switches the MOSFET on and off at a fixed frequency to control the average voltage supplied to the motors. This allows the user to control the rotational speed of the motors and thus the speed at which the Nerf darts are launched.

2.2.1.4 Sub-Circuit #4 Tank Turret Rotation

A BYJ-48 stepper motor, driven by a ULN2003 stepper motor driver, is used to precisely control the rotation of the entire turret. The idea is that the stepper motor is attached to the turret and drives a gear that is linked with a stationary sprocket on the base of the tank. By rotating the stepper motor, the entire turret will rotate along with it.

Stepper motors, known for their precision, have an open-loop control system by moving the motor shaft in small, discrete angles known as steps. According to the datasheet (<https://www.mouser.com/datasheet/2/758/stepd-01-data-sheet-1143075.pdf?srltid=AfmBOorL6QEq7SdzeHiJ8wRkhsh2NAUvU2LsQwxwKVCoroCEr7jz4MgW>), the BYJ-48 is a 4-phased motor with a 1:64 internal gear reduction, with a stride angle of 5.625°/64: each step rotates the shaft by 0.088°, with ~4096 steps for a full rotation. Unlike DC motors, stepper motors move in phases—the BYJ-48 has four

	Wave Drive				Full Step Drive				Half Step Drive							
Phase	1	2	3	4	1	2	3	4	1	2	3	4	5	6	7	8
X	1	0	0	0	1	0	0	1	1	0	0	0	0	0	1	1
Y	0	1	0	0	1	1	0	0	1	1	1	0	0	0	0	0
!X	0	0	1	0	0	1	1	0	0	0	1	1	1	0	0	0
!Y	0	0	0	1	0	0	1	1	0	0	0	0	1	1	1	0

internal coils, or electromagnets, that can be activated. Each time the right coils are powered, the rotor, which has permanent magnets, moves the shaft by one step. To achieve continuous, complete rotation, 4-phase motors run a 4-step (full) or 8-step (half) sequence to sequentially power the coils in the right order. Because of the nature of the step-by-step

movement, the stepper motor is able to achieve great positional open-loop control. By sending signals to all four of the coils, the motor is able to rotate to any position as desired, with relative accuracy.

The ULN2003 driver itself needs to power the BYJ-48, and not the Arduino, because the motor coils on the BYJ-48 draw more current than the GPIO pins on the Arduino UNO R4 can supply—the GPIO pins on the UNO R4 can only safely supply up to 8 mA, while the BYJ-48 runs on 5 V with an internal resistance of 50 Ω, resulting in a 100 mA current draw per phase

(<https://docs.arduino.cc/resources/datasheets/ABX00087-datasheet.pdf>, <https://www.mouser.com/datasheet/2/758/stepd-01-data-sheet-1143075.pdf?srltid=AfmBOorL6QEq7SdzeHiJ8wRkhsh2NAUvU2LsQwxwKVCoroCEr7jz4MgW>). The ULN2003, on the other hand, includes

transistors in a Darlington array (transistors paired together to combine their strength) to draw a higher current directly from the power supply, with each transistor handling up to 500 mA (<https://www.hadex.cz/spec/m513.pdf>), switching the motor coils safely.

To wire the circuit, the stepper motor connects its 5-pin connector directly into the ULN2003 driver board. Then, the driver connects its four IN pins to four of the Arduino D pins. To power the driver, the VIN will be connected directly to the 5 V power supply, while it must also share a common ground with the Arduino microcontroller.

ULN2003 Driver Pin	Arduino Uno R4 Pin
IN1	D8
IN2	D9
IN3	D10
IN4	D11

ULN2003 Driver Pin	Connect to
VCC	5V
GND	GND

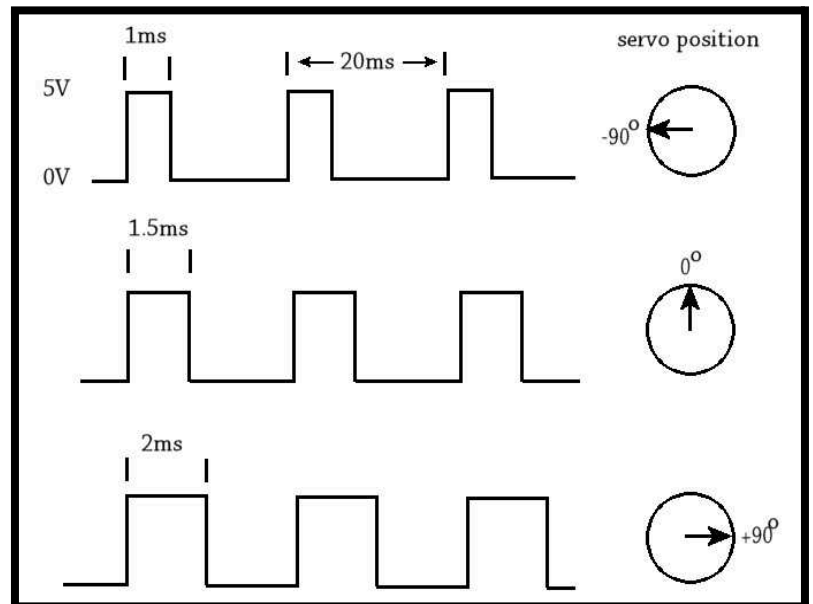
By using the Stepper library, the stepper motor can be programmed to move to precise steps by using `stepper.step(int steps)` and can be programmed to move at different speeds using `stepper.setSpeed(int speed)`. If required, the AccelStep library can be used for finer control over phasing, speed, and acceleration.

2.2.1.5 Sub-Circuit #5 Tank Dart Shover

An SG90 servo is connected to a mechanism that is able to hit nerf darts into the flywheels so that they can be launched out. The servo will be controlled by the Arduino Uno R4 microcontroller in the tank, programmed to oscillate between 0 degrees and 90 degrees whenever the user is holding down the trigger. The SG90 servo requires 3 wires: a 5 V input, a common ground, and a PWM output signal.

Because the current draw of the servo can reach up to 500 mA, the power should be fed directly from the 5 V buck converter and power supply, and the ground should be shared with the Arduino ground, while the PWM is connected to the Arduino.

The PWM control on the servo is regulated by the Servo library on the Arduino. Using `servo.write(0)` and `servo.write(90)` sends a custom PWM signal to the servo—a duty cycle of 1.5/20 tells the servo to rotate to 0°, while a duty cycle of 2/20 tells the servo to rotate to 90°—causing the internal gears in the servo to spin the final motor shaft to 0 degrees and 90 degrees, respectively, relative to the initial orientation of the internal gears. While being held, the trigger will activate an oscillation of the servo, oscillating between upright (90 degrees) and flat (0

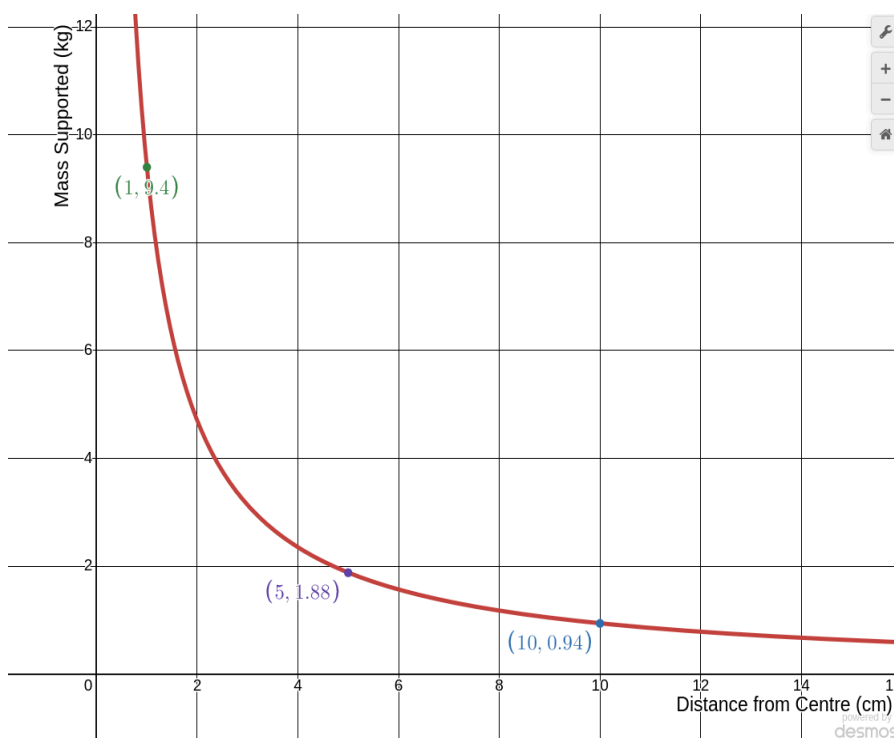


degrees) at periodic intervals that will be optimized through testing. This oscillation will allow Nerf darts to fall into the loading chamber and then get shoved into the flywheel system for shooting by a push-block mechanism attached to the servo arm, shooting automatically. Note that in the SG90 datasheet (http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf), the servo can travel at a maximum angular velocity of $60^\circ / 0.1 \text{ s}$, so a 90° rotation outlined here could theoretically take only 150 ms, or a bit more than 300 ms for a full back-and-forth cycle. However, to buffer the timing to include reversing the direction of the motor in addition to letting the Nerf dart fall into the chamber, the servo will only likely run a half-cycle at around 250 ms or a bit over 500 ms for a full cycle, allowing a max fire rate of around 2 darts per second. Additionally, once the trigger is released, the oscillation will stop, and the servo will be programmed to reset to 0 degrees to prepare for the next cycle.

2.2.1.6 Sub-Circuit #6 Tank Barrel Elevation

To elevate the barrel and attain the HRS requirement of 30 degrees, the barrel of the turret, which will be attached to the flywheel system, must be able to independently raise and lower its pitch by an MG996R servo motor. Because of mechanical constraints of shoving the nerf dart into the flywheels at a steep angle, the servo only needs to bring the barrel up and down between around $+15^\circ$ and -15° relative to the horizontal. This requires that the servo be placed horizontally and use its precision to reach those angles through PWM control. Unlike the SG90, the MG996R can only rotate 120° , from -60° to 60° , but because the requirements do not necessitate that much range of motion, the MG996R is more than enough. Mechanically, the servo will need to lift the barrel and flywheels up and down through a channel in the turret. As the [datasheet](#) mentions, the MG996R servo can hold a maximum

stall torque of $9.4 \text{ kgf} \cdot \text{cm}$ at 4.8 V, meaning that it can support a 9.4 kg load 1 cm away. Although the centre of gravity of the barrel mechanism will be further away than 1 cm, the mass of the mechanism will also be less than 9.4 kg. Even at 10 cm away, the servo will still be able to support 940 g of load; fortunately, the barrel and the flywheels are only approximately 400 g - 600 g. If the servo fails to support this load, an additional servo may be added. Like the SG90, as outlined in subcircuit 2.2.1.5, the servo requires a power signal connected directly to the power supply, the PWM signal being processed by the Servo library through the Arduino, and a common ground shared with both the power supply and the Arduino.



2.2.1.7 Sub-Circuit #7 Tank Laser Beam

The laser module improves the user's aiming accuracy while also enhancing the overall functionality and intimidation factor of the tank. A KY-008 Laser Transmitter Module is mounted on the tank barrel to indicate where the Nerf dart is expected to land. The red laser is controlled by the user through the "X" button on the Xbox controller, allowing it to be toggled on and off as needed.

The KY-008 laser module ([datasheet](#)) operates at 5 V DC and typically draws approximately 20–40 mA of current.

The module has three pins: VCC, GND, and Signal. The VCC pin is connected to the Arduino's 5 V supply, while the GND pin is connected to the common ground. The Signal pin is connected to a digital output pin on the Arduino.

When the Arduino outputs a HIGH signal, the laser is turned on; when the signal is LOW, the laser is turned off. This allows the laser to be easily controlled in synchronization with user input from the Xbox controller.

Since the current draw of the laser module can approach or exceed the recommended current limits of an Arduino I/O pin, a current-limiting resistor is included in series with the signal line to protect the microcontroller and ensure safe operation.

2.2.1.8 Sub-Circuit #8 Tank Vision Processing

The vision processing subcircuit is centred around the Raspberry Pi 4, which acts as a high-level processor for image analysis. Connected to the Pi through USB are the Arduino UNO R4 and an AverMedia PW313 webcam. The camera captures live images from the tank and sends them to the Raspberry Pi 4. Then, the Pi runs computer vision algorithms using OpenCV and other libraries to process these frames—detecting human targets in real time and finding out their position relative to the tank. Based on this analysis, the Pi generates relevant information, such as object position or targeting offsets, and transmits this data to the Arduino over a USB connection. From this data, the Arduino will use algorithms to command its actuators, controlling the autonomous mode of the tank.

Electrically, the webcam connects to the Pi through a 5 V USB-A cable, powering the webcam with less than 500 mA of current through the cable. Similarly, the Arduino connects to the Pi through a USB-C to USB-A cable, although this connection will be for data only and not power. To power the Raspberry Pi 4, continuous 5 V power will come indirectly through the power supply. As the power supply is a 7.4 V battery, an external 5 V buck converter is used to step down the voltage to power the Pi. Because the Pi is running vision processing and many calculations, it can occasionally draw substantial currents, possibly up to 2 A of current at its peaks, excluding the powering of the camera itself. The buck converter will connect to the Pi either through a powered USB-C connection or directly into the Pi's 5 V pin—USB-C is the safer option for powering the Pi but requires cutting open a USB cable and connecting the power channels to the buck converter, while powering the Pi directly through the 5 V pin bypasses any internal circuitry designed to protect the Pi.

Although this connection uses a USB cable physically, the Arduino appears to the Pi as a virtual serial device (e.g., `/dev/ttyACM0`), meaning the data is sent using a UART-style serial protocol encapsulated over USB. This allows the Pi to send structured bytes and data at a defined baud rate, which the Arduino reads using its serial interface; however, this USB connection also allows the Arduino to

transmit signals to the Pi—vision processing is only required in autonomous mode, so the Arduino can initiate vision processing when necessary on the Pi to offload power demand. The Arduino then processes this data for its programmed control logic, allowing it to activate its motors and other outputs. This method provides reliable, bidirectional communication while avoiding the voltage level issues of direct UART connections, since USB handles signal conditioning and protocol management internally.

The PW313 Webcam is connected to the Raspberry Pi 4 through a USB-A connection, which not only powers the webcam but also allows the Pi to interface with the camera using the USB Video Class (UVC protocol). When connected, the camera is recognized by the Raspberry Pi's Linux operating system as a video device (e.g., `/dev/video0`), allowing software libraries such as OpenCV to access a continuous stream of image frames. Internally, the webcam captures raw image data from its sensor, compresses it, and transmits it as high-speed USB packets to the Pi. The Raspberry Pi then processes the video, decoding the video into individual frames and then executing object detection and tracking algorithms through OpenCV. For example, humans will be detected using the MobileNet-SSD pretrained deep-learning model, and they will be tracked by a SORT (Simple Online and Realtime Tracking) algorithm to extrapolate object movement from data. To find relevant parameters of humans, like their distance or angle from the camera, the 2D bounding boxes will be extrapolated into 3D positions by assuming average human height and width for targets and using trigonometry, mapping, and camera calibration. Approximating the 3D pose of the targets will be sufficient for aiming the turret—because there is only one camera capable of detecting objects, 3D vision with depth is not exactly possible, but estimating target positions will yield some success. As such, the Raspberry Pi 4 will send basic target information like the distance and angle from the camera to the Arduino, allowing it to activate the stepper motor, servo motor, and flywheel motors to prepare to shoot at the nearest target.

2.2.2 Circuit #2 Tank-Controller Interface

2.2.2.1 Sub-Circuit #9 Tank-Controller Live Video

This sub-circuit provides a real-time First-Person View (FPV) video system that allows the operator to visually operate the tank from a remote position. It is composed of the Runcam Phoenix 2 FPV camera, the Eachine TX805 5.8GHz video transmitter (VTX), and the Sologood FPV monitor, which together form a complete live video link. The Runcam Phoenix 2 camera captures continuous analog video of the tank's surroundings and converts it into a real-time video signal. This signal is directly fed into the TX805 VTX, which transmits the video wirelessly over the 5.8 GHz frequency band. The use of this frequency allows for low-latency communication, which is essential for responsive and accurate remote driving.

The VTX is powered by the main 7.4V 2s Li-ion battery. The VTX then provides a regulated 5V output that directly powers the camera. This means the camera does not need a separate power source and instead operates through the transmitter's regulated supply line. This configuration reduces wiring complexity and ensures both devices share a consistent, filtered power source, which helps minimize video noise or signal interference caused by voltage fluctuations. The Sologood FPV monitor is powered by the rechargeable power bank through a USB line.

The FPV system operates using analog video transmission, where the camera continuously outputs a real-time electrical video signal rather than encoding it digitally. The Runcam Phoenix 2 FPV camera generates a standard analog video signal, which is sent directly to the Eachine TX805 VTX through a wired connection. This direct connection allows the signal to be transmitted with minimal processing, resulting in extremely low latency compared to digital systems. The VTX then modulates this analog

signal onto a 5.8 GHz radio frequency carrier and broadcasts it wirelessly to the FPV monitor. Because the signal is analog, any interference or signal degradation results in gradual loss of image quality rather than a complete signal drop, which is beneficial for maintaining control in real-time applications. This simple and direct connection between the camera and transmitter is key to achieving fast, reliable video feedback for responsive tank operation.

The Eachine TX805 VTX includes multiple adjustable settings that allow the video transmission to be tuned for different operating conditions. One of the main configurations is output power level, which can typically be switched between values such as 25 mW, 200 mW, 600 mW, and 800 mW. Lower power levels reduce current draw and are useful for short-range testing or minimizing heat, while higher power levels increase transmission range and signal penetration, improving reliability in outdoor or obstructed environments. The VTX also allows selection of different channels and frequency bands within the 5.8 GHz range, which helps avoid interference from other FPV systems by choosing a clean channel. To calculate the expected range, the link budget form equation is used, where Path Loss is the reduction in signal strength as a radio wave travels from a transmitter to a receiver.

$$Path\ Loss = P_t + G_t + G_r - P_r$$

P_t = Transmit Power (dBm)

G_t = Transmit Antenna Gain (dBi)

G_r = Receiver Antenna Gain (dBi)

P_r = Receiver Sensitivity (dBm)

Firstly, the power in mW needs to be converted to dBm. This is done with the formula

$$P(dBm) = 10\log(P_{mW})$$

$$p(dBm) = 10\log(600)$$

$$p(dBm) \approx 27.78\ dBm$$

The antenna on the transmitter has an estimated gain of approximately 2 dBi.

The antenna on the FPV monitor has an estimated gain of 1.5–3 dBi. For the sake of estimation, 1.5 dBi will be used for the worst-case scenario.

The FPV monitor has a super sensitive reception, which indicates a sensitivity of around -88 to -92 dBm. Again, assuming the worst-case scenario, 88dBm will be used.

$$Path\ Loss = 27.78 + 2 + 1.5 - (-88)$$

$$Path\ Loss = 119.28dB$$

Finally, the FSPL formula is used to calculate the maximum distance where

d = distance in km

f = frequency in MHz

The system uses a frequency of 5.8GHz which is equivalent to 5800 MHz

$$FSPL(dB) = 20\log(d) + 20\log(f) + 32.44$$

$$119.28 = 20\log(d) + 20\log(5800) + 32.44$$

$$11.57 = 20\log(d)$$

$$d \approx 3.78\text{km}$$

However, it is important to note that this number is theoretical and assumes a perfect line of sight, no interference, and perfect antenna alignment which is impossible in real life. In addition, because the tank is so close to the ground, ground absorption and reflection interference will also limit the maximum distance. At 5.8 GHz, signals also have poor penetration compared to lower-frequency systems, meaning the signal weakens rapidly after passing through even a small number of obstacles. Since the tank is mainly meant to be used indoors, loss due to walls, furniture, and structural materials need to be accounted for. Each wall can introduce a loss ranging from a few dB (drywall) to over 10–20 dB (brick or concrete). This brings the realistic maximum distance to around a few hundred meters depending on the type of room. However, this number is hard to estimate because of all the possible variables at play.

Overall, this sub-circuit provides a low-latency and reliable FPV system for real-time tank control by combining the FPV camera, VTX, and monitor into a simple analog video link. While calculations estimate a theoretical range of about 3.78 km, real-world factors such as ground-level operation and indoor obstacles significantly reduce this distance. Despite this, the system delivers sufficient range and stable video performance for effective short-range and indoor operation.

2.2.2.2 Sub-Circuit #10 Tank-Controller Radio

The controller-to-tank wireless communication in this project is implemented using a pair of nRF24L01 + PA + LNA devices, each interfaced to an Arduino via the Serial Peripheral Interface (SPI). At a system level, the controller-side Arduino reads user inputs, formats them into a compact digital payload, and transmits this data to the tank-side Arduino through the nRF24L01 using a 2.4 GHz radio link. The receiving module, on the tank, demodulates the signal, reconstructs the packet, and passes the payload back to the tank Arduino via SPI, where it is interpreted into motor and actuator commands.

Electrically, the nRF24L01 operates at a nominal voltage of 3.3 V. Although its average current consumption during transmission and reception is relatively low (approximately 11–14 mA in transmit mode and ~13 mA in receive mode from the [datasheet](#)), the module experiences short-duration current spikes that can exceed 100 mA, and the PA + LNA attachments draw additional current, increasing transmission current to ~125 mA and receiving current to ~45 mA. To avoid brownouts or disrupting the component's communication, an nRF24L01 Wireless Transceiver Socket Adapter Board ([datasheet](#)) will be used: it contains an onboard decoupling capacitor able to act as a "reservoir" of energy to prevent noise disruptions, and its AMS1117-3.3 regulator allows the module to connect to a 5 V input, as the Arduino can only source around ~50 mA of current through the 3.3 V pin. To wire the circuit, firstly, the adapter board has 8 female pins that plug right into the 8 male pins on the radio module. To connect the adapter board to the Arduino UNO R4, the MOSI pin connects with D11, the MISO pin connects with D12, and the SCK connects with D13. The other pins, CSN, CE, and (optionally) IRQ, plug into any digital pin (see internal

Abbreviation	Description
Cn	SPI command bit
Sn	STATUS register bit
Dn	Data Bit (Note: LSByte to MSByte, MSBit in each byte first)

Table 21. Abbreviations used in Figure 26. to Figure 28.



interfaces 2.3.1.2 and 2.3.3.4). For powering the board, it connects its VCC to the 5 V pin and also shares a common ground with the microcontroller.

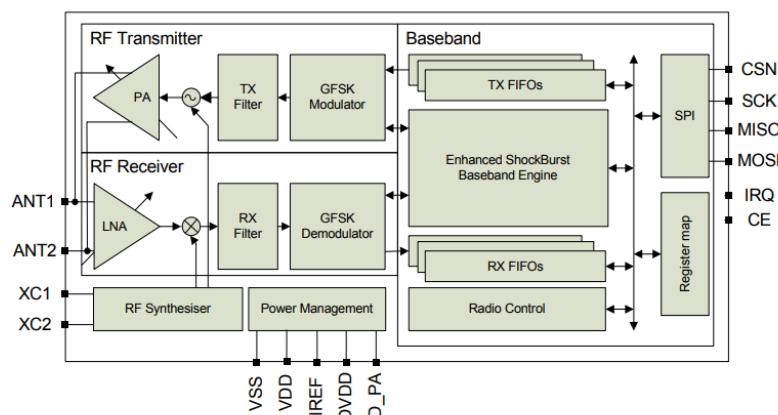
Each Arduino communicates with their respective radio modules using SPI, a synchronous, bidirectional communication protocol that connects a master microcontroller to a slave peripheral. In this case, both Arduinos maintain a 6-wire SPI connection to the slave radio modules. The MOSI (Master Out Slave In) and MISO (Master In Slave Out) signals carry data back and forth between the Arduino and the transceiver, including commands, payload data, status codes, and register data. The SCK (Serial Clock) signal synchronizes the communication; because SPI communication is run at a default frequency of 4 MHz (range from 0 to 10 MHz), sharing the same clock frequency and pulses allows the MOSI and MISO signals to be synchronized and read correctly. The CSN signal (Chip Select Not) initiates and terminates the communication—as indicated by the name, a low signal from the master begins communication, and it must be driven high to end it. Additionally, the CE (Chip Enable) pin controls the internal state of the nRF24L01: it can either be in standby, transmit, or receive mode, as a high pulse of at least 10 μ s on the CE pin initiates transmission, while holding CE high continuously places the module in receive mode after an internal settling delay of approximately 130 μ s. Once the FIFO (First In, First Out) buffer on the module contains data sent from the master and the CE (Chip Enable) pin is high, the module automatically sends the data. Finally, the IRQ (Interrupt Request) signal is used to alert the master when the radio module receives data wirelessly, letting the microcontroller read the data.

Internally, the nRF24L01+ contains a complete radio communication pipeline. Data written into the module is stored in a FIFO payload buffer. Then, the data is processed by a packet engine that automatically formats the data into a structured packet, including a preamble for clock synchronization, an address field, a packet header containing payload length and ID, the payload itself, and error checking. The radio module also uses Enhanced ShockBurst, a hardware feature that adds the preamble (start of signal), CRC math error checking (Cyclic Redundancy Check) at the end of the signal, auto-acknowledgement to respond to the transmitter that the data was received, and auto-retransmit to re-transmit the data in case the autoAck signal is not received, all of which offload processing from the Arduino microcontroller itself and reduce errors in communication. From there, a GFSK (Gaussian Frequency Shift Keying) modulator converts the digital signal into a modulated RF waveform containing the data. However, the RF Synthesizer generates the 2.4 GHz frequency, and the data-containing waveform and the 2.4 GHz radio wave are combined together, mechanically created by an oscillator, amplified by the internal PA (Power Amplifier), and transmitted via antenna. For receiving signals, the entire process is pretty much reversed, with an LNA (Low Noise Amplifier) to pick up weak radio signals, a demodulator, a frequency mixer (to bring waves back down into readable data), and a filter to process the incoming radio waves. Note that although the attached block diagram mentions the internal PA and LNA (Low Noise Amplifier), the module being used is an nRF24L01 + PA + LNA, which means that it contains an additional external PA and LNA for extra range and power.

nRF24L01+ Product Specification



1.2 Block diagram

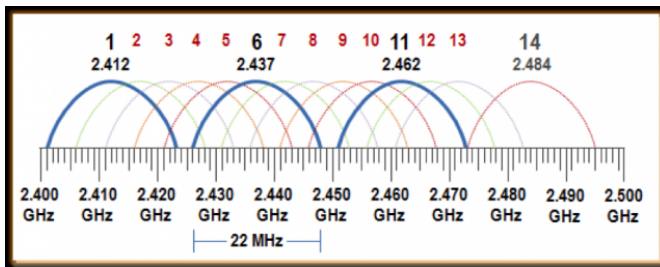


For the transmission of the signal, there are configurable features that allow for better, customizable communication. The aforementioned PA + LNA modules come with an antenna. To optimize the ability to receive and transmit signals, these antennas should be placed vertically in a high place with good visibility, clear of obstacles. If necessary, these antennas can be

upgraded to be stronger. On the GFSK modulator, the bit rate can either be transmitted at 250 kbps, 1 Mbps, or 2 Mbps. Although higher data rates sponsor faster, more responsive transmission and more complex data capacity, lower data rates allow for greater signal integrity and, thus, a longer range.

Data Rate	Sensitivity
2 Mbps	~ -82 dBm
1 Mbps	~ -85 dBm
250 kbps	~ -94 dBm

From the datasheet, the receiver sensitivity is the weakest signal the receiver can detect reliably. -94 dBm is a much weaker signal than -82 dBm, so at 250 kbps, the receiver can still work further away. The data rate of 250 kbps will likely be used for the Sentinel, as long range and signal integrity are prioritized over small differences in latency, although this will require testing. Moreover, the specific channel of the radio transmission can be selected for optimal communication



without interference. The nRF24L01+ advertises itself as being able to select from 125 channels, each spaced 1 MHz apart. This means that the modules can communicate with each other on specific frequencies, from 2.400 GHz to 2.524 GHz. However, because Wi-Fi channels 1, 6, and 11 take up zones 1-25, 26-50, and 51-73, respectively, and because any channel above 2.483 GHz is prohibited for license-free use in many countries

(along with many devices not supporting above 2.483 GHz), only channels like 76 (2.476 GHz) are legal, functional, and quiet, with limited radio interference. Additionally, the payload of the radio data can be customized. Although technically limited to 32 bytes, each payload should only contain 4-8 bytes of data—enough to transfer information and controls such as angle of turret or speed of motors, but not so much as to increase latency and interference.

2.2.3 Circuit #3 Controller

2.2.3.1 Sub-Circuit #11 Controller Power Supply

The controller-side power supply subcircuit is designed around a single portable USB power bank that provides a regulated 5 V rail to all user-interface electronics, simplifying wiring and ensuring electrical isolation from the high-current systems on the tank. The Charmast Portable Charger is used as the primary source; this device internally regulates a lithium-polymer cell (nominal 3.7 V) to a stable 5 V USB output using a boost converter, with continuous current capabilities rated at 3 A. The Arduino UNO R4 is powered directly through its USB-C port, which accepts 5 V input and distributes it to the Renesas RA4M1 MCU microcontroller and peripheral rails.

In parallel, the 5 V power bank needs to charge and power the Sologood 4.3" FPV monitor's ([datasheet](#)) internal battery through its USB-micro port, especially if the monitor is both charging and running. In normal operations, the internal lithium-ion cell provides 3.7 V at around 400 mA. However, while charging, the internal battery can draw a maximum of 500 mA, charging the internal cell from 3.0 V to 4.2 V. Because the monitor has an internal buck converter to step the 5 V input down to 3.7 V, it will undoubtedly lose some of the power during the conversion as heat, so it is assumed that the efficiency of the 5 V to 3.7 V conversion is around 85% for these simplified calculations (η = efficiency of the regulator). As demonstrated below, powering the monitor while charging it will draw about 800 mA.

$$P_{normal} = V \times I = 3.7 \text{ V} \times 0.4 \text{ A} = 1.48 \text{ W} \quad I_{normal in} = \frac{P_{normal}}{V_{in} \times \eta} \approx \frac{1.48 \text{ W}}{5 \text{ V} \times 85\%} \approx 0.35 \text{ A}$$

$$P_{\text{charging}} = V \times I = 3.7 \times 0.5 \text{ A} = 1.85 \text{ W} \quad I_{\text{charging in}} = \frac{P_{\text{charging}}}{V_{\text{in}} \times \eta} \approx \frac{1.85 \text{ W}}{5 \text{ V} \times 85\%} \approx 0.44 \text{ A}$$

$$I_{\text{combined in}} = I_{\text{normal in}} + I_{\text{charging in}} \approx 0.35 \text{ A} + 0.44 \text{ A} \approx 0.79 \text{ A}$$

Because both the Arduino and monitor are USB-powered loads, they can be connected directly from separate output ports on the power bank, avoiding the need for additional voltage regulation stages. Through the Arduino, the power bank will need to power the microcontroller itself, the nRF24L01+PA+LNA (with adapter), the USB Host Shield, and the Xbox Controller. The Arduino UNO R4 draws around 100 mA without any peripherals, depending on the USB connection. The nRF24L01+PA+LNA draws around 100 mA in transmitting mode, at its peak. The full USB Host Shield sinks around 30 mA in total (~20 mA from the MAX3421E itself). Finally, the Xbox controller draws around 125 mA while active. Note that although the nRF24L01+PA+LNA module runs on 3.3 V power, it still draws around 100 mA of 5 V current because the adapter uses a simple internal linear regulator, which only emits the extra power from stepping the 5 V down to 3.3 V as heat.

$$I_{\text{total}} = 0.79 \text{ A} + 0.100 \text{ A} + 0.030 \text{ A} + 0.125 \text{ A} = 1.045 \text{ A}$$

Overall, the total circuit is powered by 5 V and draws a total current of 1.045 A when the monitor is both running and charging. Load aggregation remains well within the safe operating region of the power bank (can source up to 3 A), even when accounting for transient spikes, which are partially mitigated by the bank's internal overcurrent and short-circuit protection circuits.

Note that due to the nature of the power bank, the power bank needs to be periodically taken off the controller and charged. The Charmast power bank has a specified capacity of 10000 mAh, meaning that if the controller system is drawing its maximum power of 1 A at 5 V, then the power bank can sustain the controller for around 10 hours. Realistically, however, current draw will not always be maximized, providing a battery life up to around 15 hours before needing to recharge it again from an external power source, like a wall plug socket, especially because the monitor has its own internal battery that can be charged.

2.2.3.2 Sub-Circuit #12 Controller Gamepad

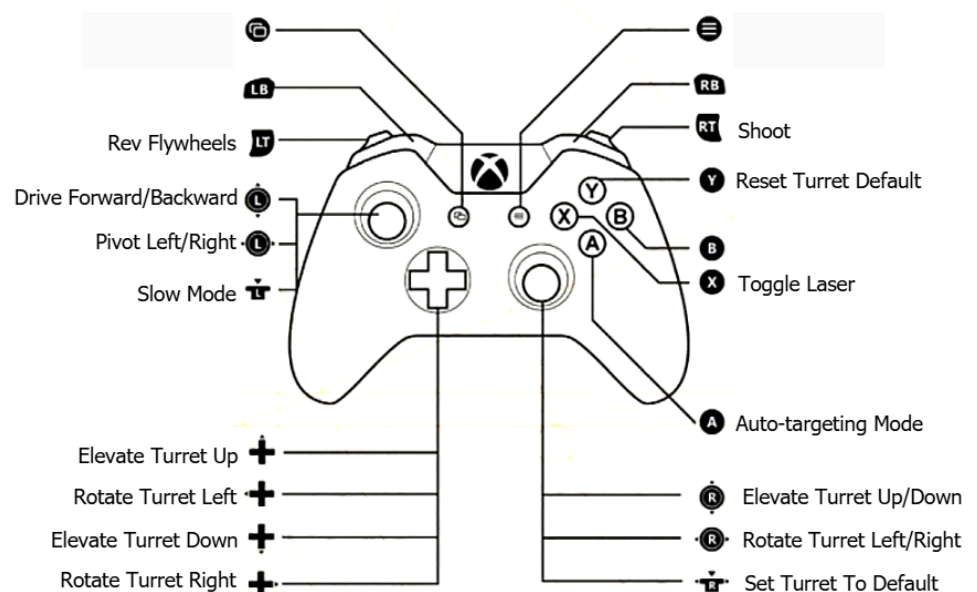
On the tank controller, the Xbox controller will be connected indirectly to the controller's Arduino UNO R4 through a USB Host Shield. Using the ARS3421U USB Host Shield ([datasheet](#)), the Arduino, using SPI, can host the wired Xbox One controller as a USB peripheral, plugged in with a USB-A to micro-USB cable. Because it is a shield, the Host Shield pins simply fit on and over the Arduino pins. However, because it uses SPI communication, the shield also contains six female pins that plug right into the ICSP header on the Arduino itself. This maintains the SPI communication, as the ICSP header contains the MISO, MOSI, SCK, GND, RESET, and VCC pins. As previously mentioned in Subcircuit 2.2.2.2, SPI is the synchronous communication protocol between a master and a slave, and they communicate using the MISO, MOSI, SCK, and CS (chip select) Signal. The CS chip, which is set to the default pin of D10, is already plugged in through the shield. On the controller-side, the Arduino will interface with two SPI devices, which presents a problem: the Arduino UNO R4 can only interface with so many pins, and there are two devices that require SPI. Since the USB Host Shield directly plugs into the ICSP SPI headers on the Arduino, the nRF24L01+PA+LNA will need to find a way to share the SPI communication of the Arduino through another method. Fortunately, the Arduino UNO R4 has default

GPIO pins that use SPI communication: D11-D13 share the MOSI, MISO, and SCK buses within the Arduino ([datasheet](#)). However, what prevents conflicts in the communication within the buses is the Chip Select (CS) system: each device has its own CS pin, and only the device whose CS is pulled LOW will respond at any given time. The USB Host Shield uses D10 as its CS (hardwired), while the radio module and its adapter will use a different pin (e.g., D8 for CSN and D7 for CE), set in the program. The Arduino switches between devices simply by selecting one CS at a time, while the others remain inactive. In this case, the controller-side microcontroller will be the master while the MAX3421E chip on the USB Host Shield will be the slave—USB inputs from the Xbox controller are read by the MAX3421E chip, and the chip then communicates with the Arduino via SPI.

On the USB side, this chip behaves like a full computer acting as a host. When a peripheral like an Xbox controller is plugged in, the chip powers the device, detects its presence through the Data+ and Data- lines, and performs a process called enumeration. During enumeration, it assigns the device an address and reads its descriptors to figure out what kind of device it is (for example, a Human Interface Device like an Xbox controller) and how it communicates. After that, the chip continuously polls the device at regular intervals, asking for updates. When the Xbox controller has new input (like a joystick movement or button press), it sends structured USB packets, which the MAX3421E receives, checks, and stores internally in its buffers and registers.

On the Arduino side, communication is much simpler and happens over SPI. The Arduino Uno R4 acts as the SPI master and talks to the MAX3421E as if it were just another peripheral. Using the USB Host Shield Library 2.0, the Arduino repeatedly performs SPI transactions to read and write the chip's registers, allowing it to check for and read new USB data—the library abstracts this process, so instead of dealing with raw USB data packets, usable values like joystick positions or button states are received.

For the gamepad itself, an Xbox One Wireless Controller will be used. The user's controls on the gamepad will be sent over USB to the USB Host Shield. On the Xbox Controller, there are two types of inputs: joysticks, which include triggers and normal joysticks, feature analogous values and axes; and buttons, which include bumpers, face buttons, and the D-pad, using digital HIGH and LOW inputs. Standard joysticks use a signed 16-bit integer (-32768 to +32767) and feature an X and Y axis. For example, on the left joystick, the centre position is (0,0), while the top-right corner would hold a value of (32767,32767). Triggers, on the other hand, are only one-directional (unsigned), use 1 axis, and can hold 8 bits of value, from 0 to 255. The digital buttons, on the other hand, are pressed and can be held at a HIGH (1) value or, when not pressed, maintain a LOW (0) value. These buttons include the face buttons A, B, X, and Y; the left and right bumpers; the buttons on the left and right joysticks themselves; the two menu buttons; and the D-Pad (Directional Pad, with switches on the top, bottom, left, and right). The control scheme being used is quite simple, as some inputs on the gamepad are left unused. The following controls will be used:



Inputs	Value Type	Description
Left Joystick	Analog Joystick, Digital Button	The left joystick is used for an arcade drive on the tank. Because the tank contains only two tracks, the tank controls its steering by independently driving each track—this is called a tank drive. Note that the tank drivetrain uses PWM to run motors at different speeds and that the motors can be reversed, allowing for turning and speed control. Arcade driving uses this to move the tank, using one axis for throttle and the other for turning. For example, moving up and down on the joystick runs both motors forward and backward (in reverse). To turn on the spot, to pivot, the joystick moving leftward along the x-axis would drive the left motor backward and drive the right motor forward, and vice versa for the right side of the joystick. A combination of the two would allow for slight turns while driving. Furthermore, smart features will be added to improve user control and responsiveness—an exponential curve will be implemented to offer smoother control with the joystick, in addition to using a deadband to prevent stick drift from affecting the driving. Likewise, the joystick button, which is activated by pressing down on the joystick, will trigger a programmed slow-mode: for fine, precise movements, the user may decide to move the tank with much slower, sensitive driving.
Right Joystick	Analog Joystick, Digital Button	The right joystick will be able to control the turret and barrel position. The turret will be able to continuously rotate $\pm 360^\circ$ and more (a slip ring helps with wiring the infinite, bidirectional, continuous rotation), while it will be able to elevate, or change its pitch by $\pm 15^\circ$ from the horizontal. To control the turret, the x-axis on the joystick will rotate the turret counterclockwise (nose left) or clockwise (nose right), and the y-axis on the joystick will be able to elevate the barrel upward (nose up) or downward (nose down). A combination of both axes would elevate the barrel while rotating it. Like the left joystick, and because both the servo (subcircuit 2.2.1.6) and stepper motor (subcircuit 2.2.1.6) include precise position and speed control, the right joystick will include a joystick exponential curve and a deadband for smoothing out the joystick movement. Additionally, pressing down on the right joystick will run a command turning the turret to reset to the default, neutral ($0^\circ, 0^\circ$) position, allowing more fine-tuned control for the user.
Left Trigger	Analog Joystick	The left trigger shall control the flywheel motors and rev them up to speed. Holding down on the joystick will drive the flywheel motors to run at PWM speeds, depending on the analog value of the trigger (0-255). Like the other joysticks, the left trigger will feature a smooth curve and a deadband to help with user control.
Right Trigger	Analog Joystick	The right trigger is responsible for shooting the darts. By pressing down the trigger, the microcontroller will send a signal to move the servo to shove the Nerf darts into the moving flywheel. Ideally, if the user holds the trigger down, the servo will oscillate and shoot the Nerf darts at a speed related to the joystick value, which will also use a deadband and a joystick curve. However, one important distinction of this system is that the programmed microcontroller will not allow the servo to move unless the flywheels are already spinning—the user cannot press the right trigger to shoot unless the left trigger is already being held or if the flywheels are being autonomously spun—because the flywheels must be spinning for the dart to be launched out; otherwise, the dart will just be pushed into a stationary wall, so this system is controlled with software logic.
Button A	Digital Button	The A face button will be used to enter an autonomous targeting mode. By holding down the A button, the tank will enable its digital camera processing that uses a trained machine vision model to find the relative poses of the closest human target(s) (within the field of vision), rotate and elevate the turret to aim at the target, and rev up the motor flywheels at desired speeds to prepare for shooting at a target. However, the tank will not actually launch the dart until the user holds down the right trigger to shoot, preventing erratic behaviour.
Button X	Digital Button	The digital X button will toggle on and off the red beam laser from subcircuit 2.2.1.7. Turning on this laser beam, which is attached to the moving turret, allows the user to shine the beam on a potential target or destination of the dart.
Button Y	Digital Button	The Y button on the gamepad will be used to reset the positions on both the servo and stepper motor to the current position of both motors. This is in case the motors slip or lose their position over time, so that setting the turret to move to the default position (see Right Trigger) can move it to the physical default, which will be set by the current position after pressing Y. Because the motors do not support overriding the initial or default position, this control can be done through software, by adding an offset to all motor positions.
D-Pad	4 Digital Buttons	The Directional Pad will be used as an extra smart feature to rotate and elevate the turret. A quick press of the left (CCW) or right (CW) pad will rotate the turret to the next 90° setpoint relative to the default 0° position. For example, if the turret is at 20° CCW to the top of the tank, quickly pressing the left pad will move the turret to 90° CCW, and pressing the left pad again will move the turret to 180° relative to the top of the tank. Likewise, quickly pressing the top or down pads will elevate the pitch of

		the barrel to the next 5° setpoint (e.g. -15°, -10°, -5°, 0°, 5°, 10°, 15°). Both of these features require pressing the pads down and letting them go quickly, for pressing and holding the pads down results in another command. If the pads are held down, the turret will rotate or elevate at a constant, continuous speed in the same direction as previously mentioned, instead of incrementing the positions by large intervals, likening the D-Pad's behaviour to the right joystick. In summary, although pressing the D-Pad down and quickly releasing it (falling edge must occur within a certain time limit) increments the turret's angular position by large intervals, holding the D-Pad down for a certain period of time (falling edge must occur outside the time limit) enables continuous rotation or elevation at a linear pace.
Other inputs not in use or for future features		

2.3 Product Interfaces

2.3.1 Controller Internal Interfaces

2.3.1.1 Interface #1: Arduino UNO R4 (U2) to USB Host Shield (U3)

From	To	Signal Name	Description
U2-JD14 (D13/SCK)	U3-SCK (4)	SPI Clock	SPI bus clock generated by the Arduino master, shared with nRF adapter on the same bus.
U2-JD12 (D11/COPI)	U3-MOSI (6)	SPI MOSI	Master-Out Slave-In data from Arduino to USB Host Shield.
U2-JD13 (D12/CIPO)	U3-MISO (5)	SPI MISO	Master-In Slave-Out data from USB Host Shield to Arduino.
U2-JD11 (D10/CS)	U3-SS (8)	Chip Select	Active-LOW select line; asserted by Arduino to address the USB Host Shield on the shared SPI bus.
U2-JD3 (D2)	U3-INT (7)	Interrupt	U3 asserts this line to notify the Arduino that a new USB HID report from the Xbox controller is ready.
+5V	U3-VCC (1)	VCC	5 V supply to the USB Host Shield.
GND	U3-GND (2)	GND	Common ground.

Overall Protocol: Standard 4-wire SPI with the Arduino as master. The USB Host Shield enumerates the Xbox controller via USB HID, parses incoming reports, and signals the Arduino via INT (D2) when new data is available. The Arduino then reads the report over SPI.

2.3.1.2 Interface #2: Arduino UNO R4 (U2) to nRF24L01 Socket Adapter Board (U4) to nRF24L01+PA+LNA (U6)

From	To	Signal Name	Description
U2-JD14 (D13/SCK)	U4-SCK (5)	SPI Clock	SPI clock shared with USB Host Shield (U3) on the same bus.
U2-JD12 (D11/COPI)	U4-MOSI (6)	SPI MOSI	Data from Arduino to nRF24L01 module.
U2-JD13 (D12/CIPO)	U4-MISO (9)	SPI MISO	Data from nRF24L01 module to Arduino.
U2-JD10 (D9)	U4-CSN (4)	Chip Select	Dedicated chip select for the nRF adapter, separate from U3-SS so both devices can share the SPI bus without conflict.
U2-JD9 (D8)	U4-CE (3)	Chip Enable	Controls RX/TX operating mode of the nRF24L01 module.
U4-SPI... (13,15,12,11,10)	U6-SPI... (1-5)	SPI	Adapter to Module SPI connection.
+5V	U4-VCC (2)	VCC	5 V supply to the nRF24L01 adapter board.
U4-VCC_3.3 (17)	U6-VDD (7)	VCC	Linear regulator converts and outputs stable 3.3 V with decoupling capacitors to U6 nRF24L01+PA+LNA
GND	U4-GND (1)	GND	Common ground.
U4-GND (16)	U6-VDD (7)	GND	Common ground.

Overall Protocol: 4-wire SPI at up to 8 Mbps. CE and CSN are independently controlled GPIOs. CE HIGH with CSN LOW initiates a TX/RX burst. The adapter board's onboard regulator provides a clean 3.3 V supply and decoupling for the nRF module.

2.3.1.3 Interface #3: Power (Battery BT1 to Arduino U2)

From	To	Signal Name	Description
BT1-+ (Charmlast Power Bank)	U2-JA5 (+5V)	Battery Supply	Power bank supplies voltage to the Arduino's 5V pin
BT1--	GND	GND	Common ground return.

Overall Protocol: Single power connection. The Charmlast power bank outputs 5 V USB, which powers the Arduino and downstream 5 V logic (USB Host Shield) and 3.3 V logic (via nRF adapter).

2.3.2 Controller External Interfaces

2.3.2.1 Interface #4: Xbox Controller (U5) to USB Host Shield (U3)

From	To	Connection	Description
U5 (Xbox Controller)	U3-USB (3)	USB	The Xbox controller connects via USB to the Host Shield, which enumerates it as a HID device and forwards input reports to the Arduino over SPI. The Xbox controller is also powered by the USB connection.

Overall Protocol: USB HID protocol. The USB Host Shield manages enumeration, polling, and report parsing. A USB-A to USB-Micro cable is used.

2.3.1.2.2 Interface #5: Arduino Programming (U2)

From	To	Connection	Description
Computer USB port	U2 Arduino USB port	USB	Used to upload firmware and debug via the Arduino IDE Serial Monitor.

Overall Protocol: Arduino bootloader over USB-serial bridge. Standard USB-A to USB-C cable.

2.3.3 Tank Internal Interfaces

2.3.3.1 Interface #6: Arduino UNO R4 (U11) to Motor Drivers (U14, U17 — DRV8871DDA)

From	To	Signal Name	Description
U11-JD6 (D5)	U14-IN1 (3)	Motor A Direction 1	Logic input 1 for DRV8871 H-bridge driving M4 (37 mm DC).
U11-JD8 (D7)	U14-IN2 (2)	Motor A Direction 2	Logic input 2 for M4. IN1/IN2 together set direction, coast, or brake.
U11-JD3 (D2)	U17-IN1 (3)	Motor B Direction 1	Logic input 1 for second DRV8871 driving M5 (37 mm DC).
U11-JD5 (D4)	U17-IN2 (2)	Motor B Direction 2	Logic input 2 for M5.
U14-OUT1 (6) / OUT2 (8)	M4-+ / M4--	Motor Output	H-bridge output to 37 mm DC motor M4 terminals.
U17-OUT1 (6) / OUT2 (8)	M5-+ / M5--	Motor Output	H-bridge output to 37 mm DC motor M5 terminals.
BT2 (via F1, SW1)	U14-VM (5) / U17-VM (5)	Motor Supply	Battery voltage (2S Li-ion) feeds the motor supply pin of each driver.

Overall Protocol: The DRV8871DDA takes two logic-level inputs (IN1, IN2) to control direction and braking. IN1 HIGH / IN2 LOW drives forward; reversed inputs drive reverse; both LOW coasts; both HIGH brakes. No PWM speed control signal is shown—speed would be modulated by applying PWM to IN1 or IN2 in firmware.

2.3.3.2 Interface #7: Arduino UNO R4 (U11) to ULN2003 Stepper Driver (U8) and BYJ-48 Stepper Motor (M2)

From	To	Signal Name	Description
U11-JA9 (A0/D14)	U8-I1 (1)	Stepper Phase 1	Arduino drives input 1 of ULN2003; output O1 (16) switches stepper coil 1 of M2 (28BYJ-48).
U11-JA10 (A1/D15)	U8-I2 (2)	Stepper Phase 2	Drives coil 2.
U11-JA11	U8-I3 (3)	Stepper	Drives coil 3.

(A2/D16)		Phase 3	
U11-JA12 (A3/D17)	U8-I4 (4)	Stepper Phase 4	Drives coil 4.
U8-O1...O4 (16–13)	M2 coil pins	Coil Drive	Low-side open-collector outputs for the 28BYJ-48 coil terminals, connecting 5V to GND
+5V	U8-VCC (9), M2-1	Coil Supply	5 V supply for the stepper coils

Overall Protocol: The Arduino sends a 4-step sequence on D14–D17 to step the 28BYJ-48 in half- or full-step mode. The ULN2003 Darlington array sinks current through each coil, powering each coil in sequence.

2.3.3.3 Interface #8: Arduino UNO R4 (U11) to Servo Motors (M1 — SG90, M3 — MG996R)

From	To	Signal Name	Description
U11-JA13 (D4/D18)	M1-CTRL (1)	Servo PWM	50 Hz PWM signal; 1–2 ms pulse width sets angular position of SG90 (M1).
U11-JA14 (A5/D19)	M3-CTRL (1)	Servo PWM	50 Hz PWM signal for MG996R (M3).
+5V	M1-+ (2) / M3-+ (2)	VCC	5 V servo power supply from buck converter (U16).
GND	M1-- (3) / M3-- (3)	GND	Common ground.

Overall Protocol: Standard RC servo PWM protocol at 50 Hz. Pulse width between 1 ms (0°) and 2 ms (180°) controls shaft angle for SG90. Arduino Servo Library is used to generate the signal.

2.3.3.4 Interface #9: Arduino UNO R4 (U11) to nRF24L01 Adapter Board (U13) to nRF24L01+PA+LNA (U15)

From	To	Signal Name	Description
U11-JD14	U13-SCK (5)	SPI Clock	SPI clock generated by Arduino master and

(D13/SCK)			shared with nRF24L01.
U11-JD12 (D11/COPI)	U13-MOSI (6)	SPI MOSI	Data from Arduino to nRF24L01 module.
U11-JD13 (D12/CIPO)	U13-MISO (9)	SPI MISO	Data from nRF24L01 module to Arduino.
U11-JD9 (D8)	U13-CSN (4)	Chip Select	Dedicated chip select for the nRF adapter
U11-JD11 (D10)	U13-CE (3)	Chip Enable	Controls RX/TX operating mode of the nRF24L01 module.
U13-SPI... (13,15,12,11,10)	U15-SPI... (1-5)	SPI	Adapter to Module SPI connection.
+5V	U13-VCC (2)	VCC	5 V supply to the nRF24L01 adapter board.
U13-VCC_3.3 (17)	U15-VDD (7)	VCC	Linear regulator converts and outputs stable 3.3 V with decoupling capacitors to U6 nRF24L01+PA+LNA
GND	U13-GND (1)	GND	Common ground.
U13-GND (16)	U15-VDD (7)	GND	Common ground.

Overall Protocol: 4-wire SPI. CE and CSN are independently controlled. The adapter board provides onboard decoupling capacitors for the nRF module.

2.3.3.5 Interface #10: Arduino UNO R4 (U11) to Size-130 DC Motors (M6, M7) via MOSFET (Q1)

From	To	Signal Name	Description
U11-JD10 (D9)	R1-1 (220 Ω) → Q1-G (1)	Gate Drive	Arduino drives the IRLZ44N gate through a 220 Ω series resistor to switch both Size-130 motors.
R2 (10 k Ω)	Q1-G (1) to GND	Gate Pull-down	Ensures gate is held LOW (motor off) when the Arduino pin is high-impedance or unpowered.

Q1-D (2)	M6-- (2) / M7-+ (1)	Motor Common Drain	MOSFET drain connects to the negative and positive terminals, respectively, of M6 and M7, switching both motors together.
Q1-S (3)	GND	Source	MOSFET source to ground.
D1 / D2 (SS24)	Across M6 / M7	Flyback Protection	Signal diodes placed across each motor terminal to suppress inductive voltage spikes when switching off.
+7.4V	M6-+ (1) / M7-- (2)	VCC	7.4V power into M6 and M7.

Overall Protocol: The IRLZ44N is a logic-level N-channel MOSFET that fully saturates with a 3.3 V gate drive. M6 and M7 run in reverse, switching together as one load through the MOSFET. The 220 Ω gate resistor limits inrush current; the 10 k Ω pull-down ensures a defined off-state.

2.3.3.6 Interface #11: Arduino UNO R4 (U11) to Laser Module (LD1 — KY-008)

From	To	Signal Name	Description
U11-JD7 (D6)	LD1-S (1)	Laser Enable	Digital output; HIGH fires the laser diode, LOW extinguishes it.
+5V	LD1-+ (2)	VCC	5 V supply to the KY-008 module.
GND	LD1-- (3)	GND	Common ground.

Overall Protocol: Simple digital on/off. The KY-008 module includes an onboard current-limiting resistor for the laser diode.

2.3.3.7 Interface #12: Arduino UNO R4 (U11) to Raspberry Pi 4 (U7)

From	To	Signal Name	Description
U11-1 Arduino USB Port	U7-41 Raspberry Pi USB Port	Serial USB	Onboard USB interface (USB CDC / Serial over USB) enables serial data transfer at an agreed baud rate.

Overall Protocol: USB CDC (Virtual Serial Port over USB). The Arduino UNO R4 enumerates as a USB serial device when connected to the Raspberry Pi. Communication occurs through the operating system's /dev/ttyACM0 (or similar) interface. A standard USB-A to USB-C cable will be used.

2.3.3.8 Interface #13: Power Circuit (BT2 → F1 → SW1 → Distribution)

From	To	Signal Name	Description
BT2-+ (2S Li-ion)	F1-1	Battery Supply	Raw 2S Li-ion voltage (~7.4 V) fed into the 15 A fuse for overcurrent protection.
F1-2	SW1-A (1)	Fused Supply	Fused battery voltage routed to the power switch.
SW1-B (2)	U16-IN+ (1), U14-VM (5), U17-VM (5), M5-+ (1), M7-- (2), U9-VCC (1)	Switched Supply	Switched battery voltage distributed to the buck converter, motor drivers, size-130 motors, and VTX.
U16-OUT+ (3)	U11-JA5 (5V), U7-2 (5V), U8-VCC (17), M1-+ (2), M2-+ (2), U13-VCC (2), LD1-+ (2)	Regulated 5 V	Buck converter (XL4016, U16) steps down battery to 5 V for logic subsystem and low-voltage actuators.
BT2--	GND	GND	Shares common ground with entire circuit

Overall Protocol: Series protection chain: battery → fuse → rocker switch → load. The 15 A fuse (F1) protects against short circuits on the motor supply. SW1 provides a single master power cut-off. The XL4016 (U16) provides a regulated 5 V rail for the Arduino, Raspberry Pi, stepper driver, servos, and laser, while the battery supplies a 7.4 V rail to the drivetrain drivers, flywheel motors, and camera subsystem.

2.3.3.9 Interface #14: FPV Camera (U12) and VTX Transmitter (U9)

From	To	Signal Name	Description
+7.4V	U9-VCC (1)	VCC	7.4 V supply from Li-ion 2S to VTX Eachine TX805
U9-VCC_C AM (3)	U12-VCC (1)	VCC	7.4 V is internally stepped down to 5 V in the VTX Eachine TX805 to power the Runcam Phoenix 2
GND	U9-GND (2),	GND	Common ground

	U12-GND (2)		
U12-VIDEO O (3)	U9-VIDEO (5)	Composite Video	Analog composite video signal passed from the Runcam Phoenix 2 to the Eachine TX805 for wireless broadcast.

Overall Protocol: Analog composite video. The FPV chain is powered from the 7.4 V rail. The VTX broadcasts on a selected 5.8 GHz channel.

2.3.4 Tank External Interfaces

2.3.4.1 Interface #15: Raspberry Pi 4 (U7) — Webcam (U10)

From	To	Connector	Description
U7 (Raspberry Pi 4)	U10 (PW313 Webcam)	USB	Webcam connects to a USB port on the Raspberry Pi for live video capture.

Overall Protocol: USB Video Class (UVC) for the webcam.

2.3.4.2 Interface #16: Arduino Programming (U11)

From	To	Connector	Description
Computer USB port	U11-41 Arduino USB port	USB	Firmware upload and serial debugging via the Arduino IDE.

Overall Protocol: Arduino USB-serial bootloader. Standard USB-A to USB-B cable.

2.3.4.3 Interface #17: Raspberry Pi Programming (U7)

From	To	Connector	Description
Computer	Pi USB-C Port (Power/Data)	USB	"USB Gadget" data link for SSH/VNC.

Computer	Pi Network Interface	Wi-Fi / Ethernet	Wireless or wired programming via SSH, VNC, or IDE.
----------	----------------------	------------------	---

Overall Protocol: Secure Shell (SSH) or SFTP established over a TCP/IP network connection, through which a remote terminal or file transfer session is maintained. This allows code to be authored on a primary workstation and subsequently executed or synchronized directly within the Raspberry Pi's native Linux filesystem.

2.3.5 Controller-Tank Interfaces

2.3.5.1 Interface #18: FPV Video Downlink → FPV Monitor

From	To	Connection	Description
U9 (Eachine TX805 VTX, Tank)	U1 (FPV Monitor, Controller)	5.8 GHz RF	Analog FPV video broadcast from tank to the monitor on the controller side.

Overall Protocol: Analog 5.8 GHz FPV. The transmitter and monitor must be set to the same channel. Signal quality depends on RF line-of-sight conditions and the selected power of the VTX.

2.3.5.2 Interface #19: nRF24L01 Wireless Link

From	To	Connection	Description
U6 (nRF24L01, Controller)	U15 (nRF24L01, Tank)	2.4 GHz RF	Bidirectional wireless link carrying joystick/button commands from controller to tank and ACK packets in return.

Overall Protocol: nRF24L01+ Enhanced ShockBurst™ over 2.4 GHz. Packets include preamble, address field, up to 32-byte payload, and CRC. The controller acts as transmitter and the tank as primary receiver. Hardware auto-acknowledgement and auto-retransmit are supported.

3. Scheduling

3.1 Tasks

Design and Ordering:

1. Obtain parts: Research and order all required components (Arduino, Raspberry Pi, motors, FPV

camera, VTX, battery, buck converter, etc.).

2. Design the tank structure: Create a CAD model of the chassis, mounts for motors, electronics, and FPV system.
3. Design the internal layout: Plan placement of battery, Arduino, Raspberry Pi, buck converter, and wiring paths.
4. Design the circuitry: Create schematics for power distribution, motor drivers, sensors, and communication modules.
5. Design FPV system integration: Plan mounting and wiring of the FPV camera and video transmitter.
6. Design controller system: Plan how the user will control the tank (USB controller, laptop interface, etc.).

Building and Assembly:

7. 3D print the tank chassis and mounts: Print all required structural components.
8. Assemble drivetrain: Install DC motors, wheels/tracks, and motor driver.
9. Build power system: Connect battery, buck converter, voltage regulation, and power switch.
10. Wire Arduino subsystem: Connect motor driver, sensors, and communication modules.
11. Wire Raspberry Pi subsystem: Connect USB camera or communication interfaces.
12. Build FPV system: Connect FPV camera to VTX and power supply.
13. Install cooling system: Add and mount fan if required for heat management.
14. Organize wiring: Secure and manage cables for reliability and safety.

Coding and Integration:

15. Program motor control: Write Arduino code to control movement (forward, backward, turning).
16. Program communication system: Enable communication between controller and tank (USB/Bluetooth/WiFi).
17. Program Raspberry Pi: Set up video streaming or processing if using a digital camera.
18. Test FPV transmission: Ensure video signal is transmitted and received correctly.
19. Integrate systems: Ensure Arduino, Raspberry Pi, and FPV systems work together properly.

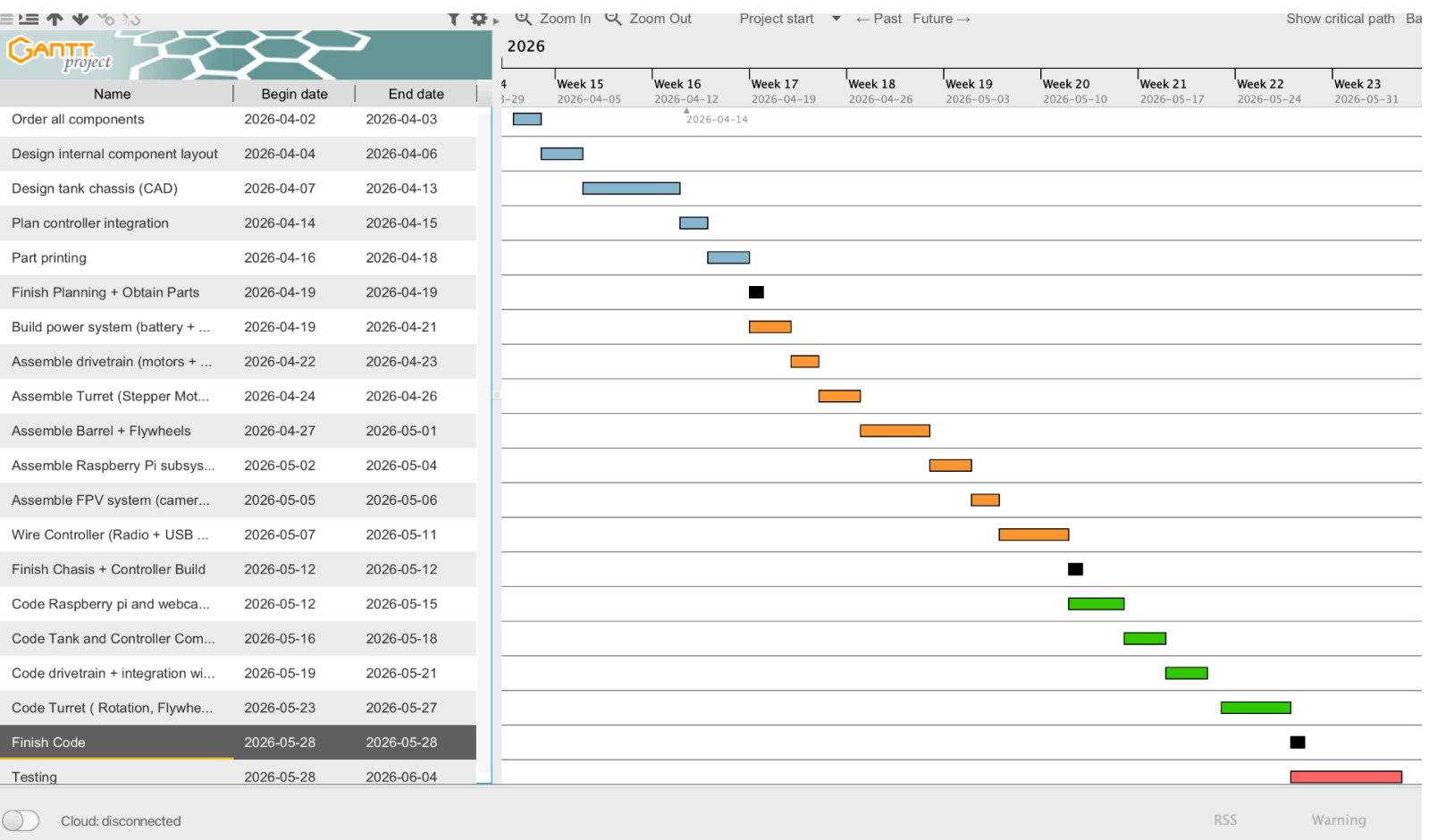
Testing and Optimization:

20. Test movement: Verify motor control, steering, and responsiveness.
21. Test power system: Check voltage stability and current draw under load.
22. Test FPV system: Verify video range, clarity, and latency.
23. Troubleshoot issues: Fix wiring, code bugs, overheating, or signal problems.
24. Optimize performance: Improve efficiency, responsiveness, and durability.

Finalization:

25. Final assembly: Secure all components inside the chassis.
26. Final testing: Perform full system test under real operating conditions.
27. Documentation: Record design, schematics, and code for submission.
28. Presentation preparation: Prepare explanation of system design and functionality.

3.2 Gantt Chart Schedule



Appendix B. Parts List

Ref.	Qty.	Part	Description	ID	Cost	Source
BT1	1	Charmast Power Bank	10000 mAh Powers FPV monitor and Arduino	W1148	\$33.99	Owened Amazon
BT2	1	2S Li-ion Battery	7.4 V, 5000mAh Powers all components on the tank	18650	\$10.88	AliExpress
D1,D2	2	SS24 Schottky Barrier Diode	40 V, 2 A Used with flywheel motors to protect MOSFET from back EMF	SSA24	2x \$0.41	Digikey
F1	1	15A Fuse	Breaks the circuit if max current is reached to prevent damage to components	N/A	\$6.13	Amazon (Includes Holder)
LD1	1	KY-008 Laser Transmitting Module	Shines a red dot laser to indicate where the dart will hit when shot	KY-008	\$4.11	Owened Amazon
M1	1	SG90 Servo	Shoves Nerf darts into flywheels to be shot out	SG90	\$9.34	Owened Amazon
M2	1	BYJ-48 Stepper Motor	Rotates tank turret	BYJ-48	\$7.49	Owened Amazon
M3	1	MG996R Servo	Elevates barrel up and down	MG996R	\$14.25	Amazon
M4,M6	2	JGB37-550 37mm DC	Motors used to turn tank treads.	JGB37-550	2x \$33.63	Amazon

M5,M7	2	Nerf Size-130 DC	Flywheel motors used to spin flywheels	N/A	N/A	Owned Comes from Rayvenfire Nerf Blaster: Discontinued
Q1	1	IRLZ44N MOSFET	Acts as an electronic switch to control flywheels	IRLZ44N	\$2.76	Digikey
R1	1	220Ω	Limits the flow of electric current in a circuit, helping protect components.	N/A	\$11.99	Owned Walmart 100 pack
R2	1	10KΩ	Limits the flow of electric current in a circuit, helping protect components	N/A	\$4.27	Owned Amazon 20 pack
SW1	1	Toggle SPST	Toggle switch to break or connect the circuit	WPC-06+ KN3C-101	\$12.55	Amazon
U1	1	Sologood FPV Monitor	Displays live video from the camera	N/A	\$59.99	Sologood
U2,U11	2	Arduino UNO R4	One Arduino on tank that controls physical tank components One Arduino that communicates with the XBox controller and sends signal to the tank through a radio module	ABX0008 7	2x \$43.33	Owned Amazon
U3	1	USB Host Shield	Plugs into Arduino to enable USB communication	N/A	\$23.95	PiShop
U4,U13	2	nRF24L01 Socket Adapter Board	Makes it easier to connect and power an nRF24L01 wireless module by providing stable voltage regulation	N/A	2x \$1.64	AliExpress

			and convenient pin connections to a microcontroller.			
U5	1	Xbox One Controller	Controller used to control the many functions of the tank	N/A	\$69.96	Owned Walmart
U6,U15	2	nRF24L01+PA+LNA Radio Module	Enable radio communication between the tank chassis and Xbox controller	NRF24L01	\$5.68	Owned Amazon
U7	1	Raspberry Pi 4	Connects to webcam for image processing	SC0193	\$109.90 (With SD Card)	Owned PiShop
U8	1	ULN2003 Stepper Motor Driver	Converts low-power control signals into high-current pulses to move the motor in precise steps.	ULN2003	\$21.00	Owned Walmart
U9	1	VTX Transmitter Eachine TX805	Transmits live analog video from FPV camera to FPV monitor	N/A	\$49.38	AliExpress
U10	1	AverMedia PW313 Webcam	Connects to Raspberry pi for video processing	N/A	\$89.99	Owned Staples
U12	1	Runcam Phoenix 2 FPV Camera	Captures live analog video	N/A	\$29.99	RunCam
U14,U17	2	DRV8871DDA DC Motor Driver	Motor driver used to control direction and speed of tank tread motors	DRV8871DDA	2x \$5.19	Amazon
U16	1	XL4016 Buck Converter	Provides 5V for components that require a max of 5V	XL4016E1	\$6.50	Amazon
Other parts not featured in the schematic are not included. The actual product contains features like tank trunks,						

3D-printed parts, a slip ring, and more.
--

Appendix C. References

Datasheets & Component References

- <https://www.farnell.com/datasheets/1863913.pdf>
- <https://www.programmingelectronics.com/r4-wifi/>
- https://www.kjell.com/globalassets/mediaassets/701916_87897_datasheet_en.pdf
- https://www.handsontec.com/dataspecs/motor_fan/MG996R.pdf
- <https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-modelb/documents/RP008341-DS-1-raspberry-pi-4-datasheet.pdf>
- <https://www.avermedia.com/product-detail/PW313>
- <https://www.rcgroups.com/forums/showthread.php?3127785-Review-of-Eachine-TX805-5-8G-40-CH-25-200-600-800mW-FPV-Transmitter-w-Smart-Audio>
- https://www.runcam.com/download/Phoenix2/Phoenix_2_Manual.pdf
- <https://www.ic-components.com/blog/28BYJ-48-Stepper-Motor-Wiring.Speed.and-Applications.jsp>
- <https://www.espbboards.dev/sensors/ky-008/>
- <https://www.mouser.com/datasheet/2/758/stepd-01-data-sheet-1143075.pdf>
- <https://docs.arduino.cc/resources/datasheets/ABX00087-datasheet.pdf>
- <https://www.hadex.cz/spec/m513.pdf>
- http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sq90_datasheet.pdf
- https://cdn.sparkfun.com/assets/3/d/8/5/1/nRF24L01P_Product_Specification_1_0.pdf
- <https://www.makerstore.com.au/product/nrf24l01-wireless-transceiver-socket/>
- <https://www.elecrow.com/wiki/usb-host-shield-for-arduino.html>
- https://en.wikipedia.org/wiki/Frequency_synthesizer
- <https://en.wikipedia.org/wiki/List%20of%20WLAN%20channels?oldformat=true>